Product Requirement Document (PRD)

Product Name: AI Concierge & Booking Assistant

1. Overview

The AI Concierge & Booking Assistant is a **multimodal agent** that supports:

- **Realtime Voice Conversations** (OpenAI Realtime API)
- **Chatbot Text Interface** (normal endpoints for mobile app)

It helps users **search, recommend, and book vacation homes**. The backend uses **Supabase/Postgres** for structured storage and integrates with **Stripe** + **WhatsApp API** for payments.

The system supports:

- **Property Recommendations** (RAG-enabled search)
- **Bookings** (user + property association, payments)
- **Check-in/Check-out flow**
- **FAQs & Concierge services**

2. Goals & Objectives

1. Provide **personalized property recommendations** in both **voice** and **chat** mode.
2. Support **end-to-end bookings** with user details, payment links, and confirmations.
3. Enable **status management** for check-in and check-out.
4. Provide a **FAQ concierge service** for common queries.
5. Make the solution **accessible across platforms** (voice agent + mobile app chatbot).

3. Target Users

- **Travelers / Guests** booking vacation rentals.
- **Hosts / Admins** monitoring property bookings.

4. Functional Requirements

4.1 Property Recommendation

- Users can filter by budget, amenities, or location.
- **RAG-powered search** from Supabase DB.
- Available via:
    - **Realtime voice agent**
    - **Chatbot API** (mobile app text UI).

4.2 Booking Flow

- Collect user details (name, phone, email).
- Store booking (`user_id`, `property_id`) in DB.
- Status flow: `pending → confirmed → checked_in → checked_out`.
- Payment flow:
    - Stripe payment link generated.
    - Sent via **WhatsApp API**.

4.3 Check-In / Check-Out

- Users can initiate check-in/checkout via **voice** or **chat**.
- Update booking status in DB.

4.4 FAQs / Concierge Services

- FAQs stored in DB.
- If no match → fallback to LLM.
- Available via:
    - **Voice agent** (spoken response).
    - **Chatbot API** (text reply).

5. Database Schema (Supabase/Postgres)

No changes — still includes:

- `users`
- `properties`
- `bookings`
- `faqs`

6. Integrations

- **OpenAI Realtime API** → Voice conversations.
- **OpenAI Chat Completions API** → Chatbot for mobile app.
- **Supabase** → Database (users, properties, bookings).
- **Stripe** → Payments.
- **WhatsApp API** → Notifications + payment links.

7. User Flows

Flow 1: Property Search & Booking (Voice + Chat)

1. User: *"Find me a villa in Miami with a pool under $300/night."*
2. Assistant fetches results via **RAG + DB query**.

3. Assistant recommends options.
4. User selects property → Assistant asks for details.
5. Booking created → Stripe link sent via WhatsApp.
6. After payment → status updates to `confirmed`.

Flow 2: Check-In / Check-Out

1. User (voice or chat): *"I want to check in."*
2. Assistant verifies booking.
3. Updates status to `checked_in`.
4. Later: *"I want to check out."*
5. Updates status to `checked_out`.

Flow 3: FAQs (Voice + Chat)

1. User: *"What's the WiFi password?"*
2. Assistant queries `faqs` table.
3. Returns direct answer or fallback to LLM.

8. Chatbot Endpoints (Mobile App)

We'll expose standard **REST API endpoints** that the mobile app can call:

- `POST /chat/message` → Send user message, get AI response.
- `POST /properties/search` → Search properties by filters.
- `POST /booking/create` → Create booking.
- `POST /booking/update-status` → Update status.
- `GET /faq` → Get FAQ answer.

The chatbot API internally calls the same **function definitions** as the voice agent, ensuring consistent logic.

9. Non-Functional Requirements

- **Cross-platform:** Voice (Realtime) + Text (Mobile app).
- **Scalable:** Handle many users simultaneously.
- **Secure:** PII and payments stored securely.
- **Consistent UX:** Chatbot and Voice use the same backend logic.